



Alma Mater Studiorum-Università di Bologna

Scuola di Ingegneria

Fondamenti di Informatica T2

Esercitazione Individuale: JUnit

Corso di Laurea in Ingegneria Informatica

Anno accademico 2023/2024

Prof. ENRICO DENTI

Dipartimento di Informatica – Scienza e Ingegneria (DISI)

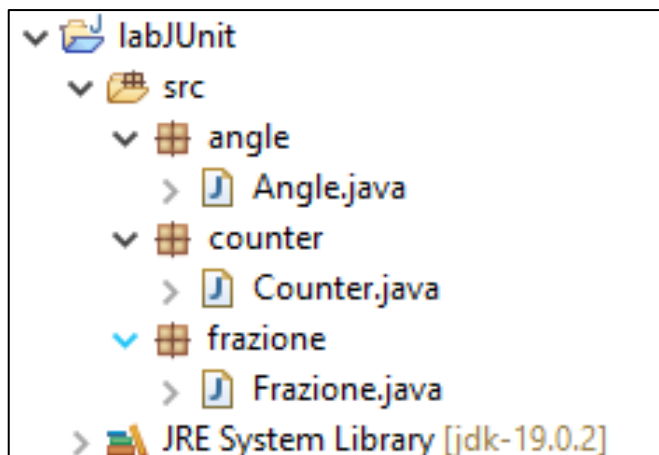


Scrivere test

- Nelle scorse lezioni abbiamo sviluppato le classi **Counter, Angle, Frazione**
- All'epoca, il collaudo fu fatto artigianalmente
 - un mini-main
 - uso diretto di **assert** nel codice
- Nuovo obiettivo:
collaudarle «seriamente» con JUnit

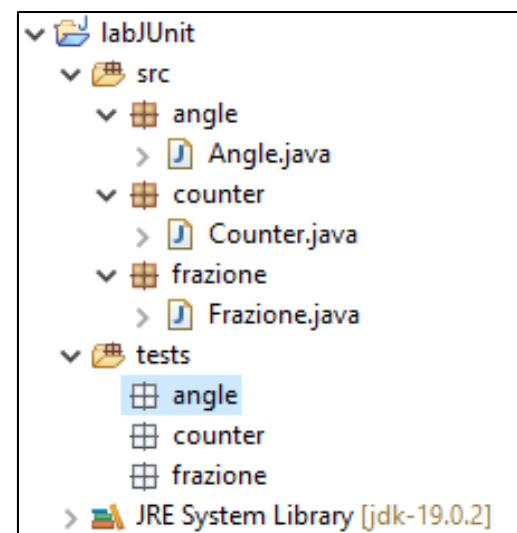
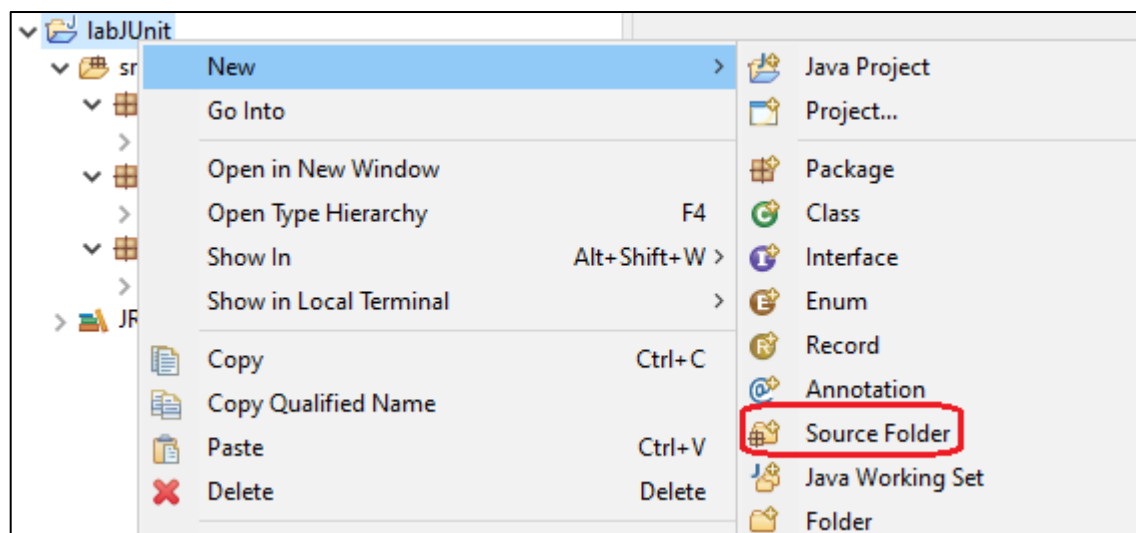
Struttura del progetto

- Innanzitutto, recuperiamo le classi **Counter**, **Angle**, **Frazione** e **disponiamole in appositi package**, evitando l'uso del package di default



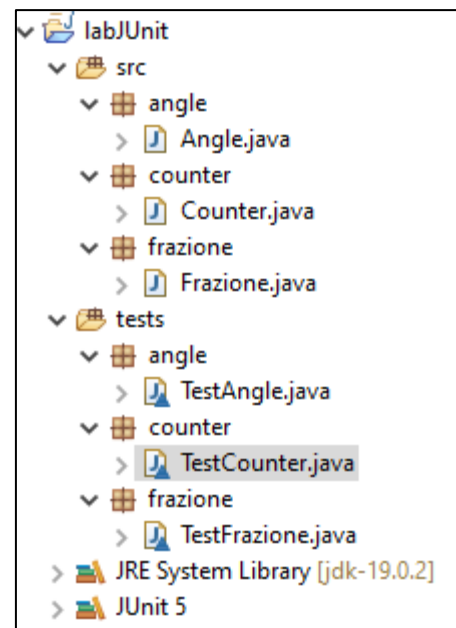
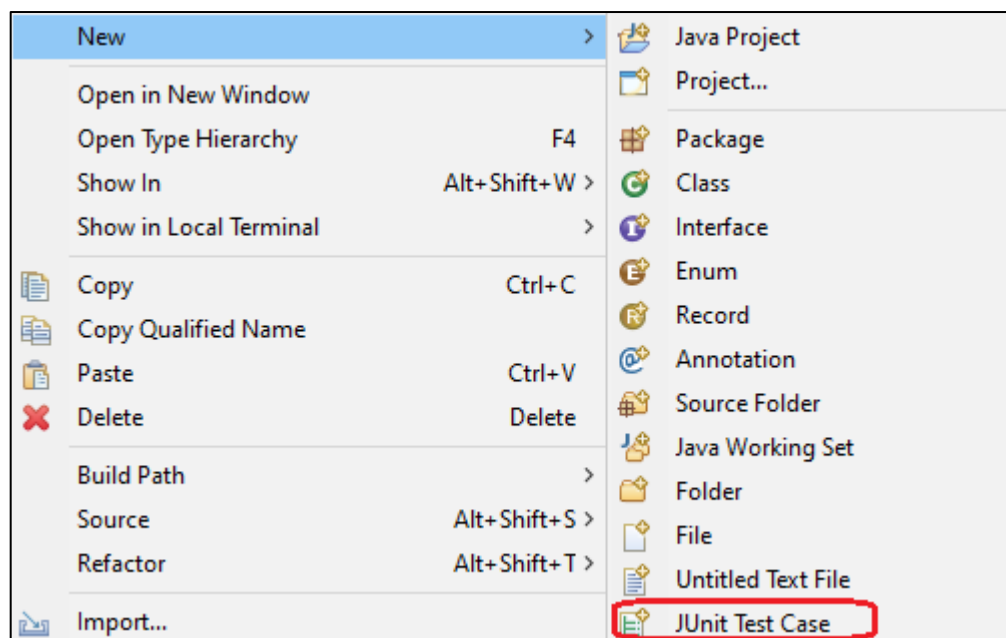
Struttura del progetto

- Poi, per mantenere ordinato il tutto
 - aggiungiamo al progetto **un'altra source folder, tests**
 - prevediamo in essa **altrettanti package**, uno per classe



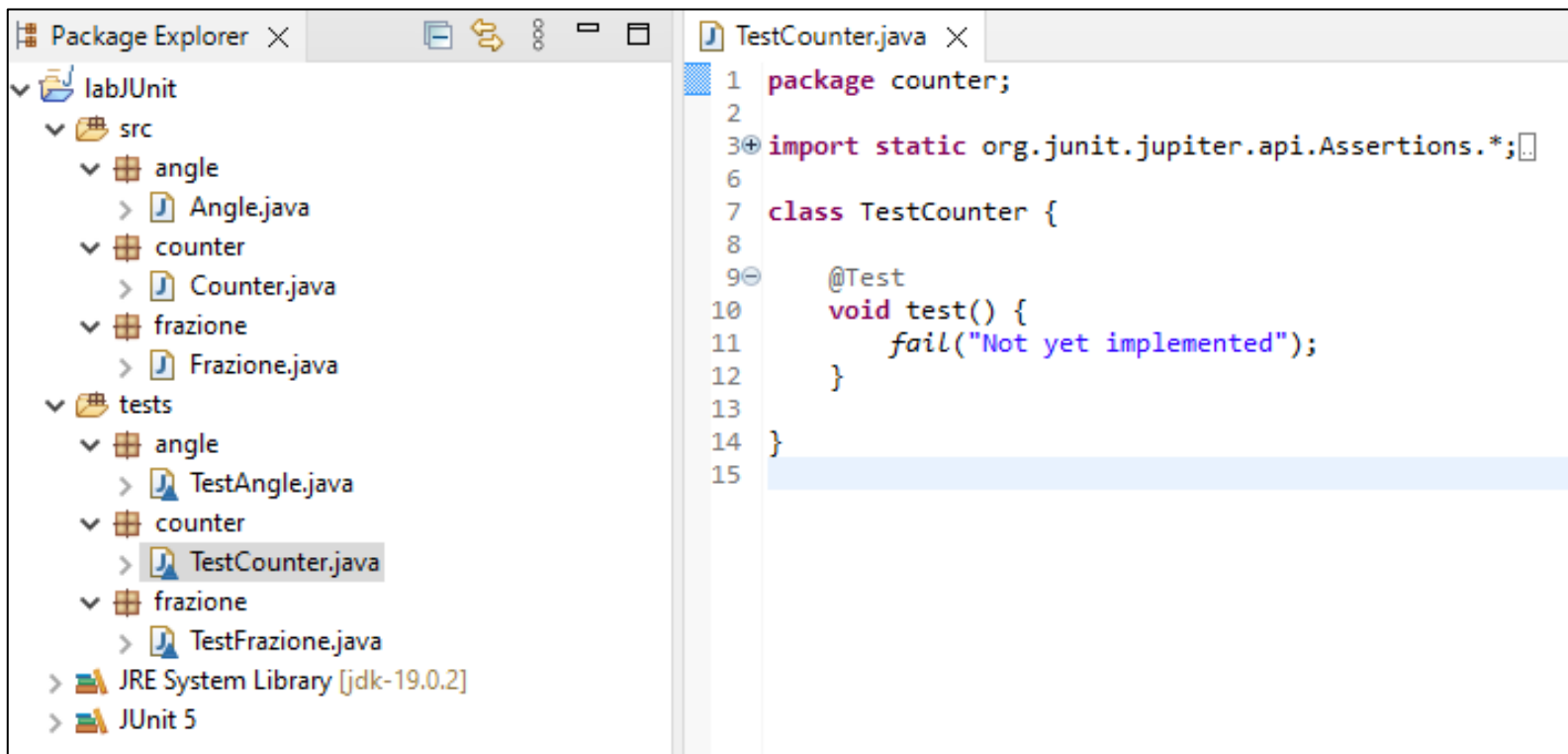
Struttura del progetto

- In questi nuovi package creiamo le classi di test
 - aggiungiamo al progetto **un'altra source folder, tests**
 - prevediamo in essa **altrettanti package**, uno per classe



Struttura del progetto

- Il wizard avrà anche creato gli scheletri di tali classi, contenenti un unico test «non yet implemented»



The screenshot shows an IDE with two panes. The left pane, labeled 'Package Explorer', displays the project structure. The right pane shows the code for 'TestCounter.java'.

Package Explorer Structure:

- labJUnit
 - src
 - angle
 - Angle.java
 - counter
 - Counter.java
 - frazione
 - Frazione.java
 - tests
 - angle
 - TestAngle.java
 - counter
 - TestCounter.java
 - frazione
 - TestFrazione.java
 - JRE System Library [jdk-19.0.2]
 - JUnit 5

TestCounter.java Code:

```
1 package counter;
2
3 import static org.junit.jupiter.api.Assertions.*;
4
5
6
7 class TestCounter {
8
9     @Test
10     void test() {
11         fail("Not yet implemented");
12     }
13
14 }
15
```

Let's test

- Scriviamo uno o più test per ogni costruttore/metodo

```
class TestCounter {  
  
    @Test  
    void testConstructor0() {  
        Counter c = new Counter();  
        assertEquals(1, c.getValue());  
    }  
  
    @Test  
    void testConstructor1a() {  
        Counter c = new Counter(10);  
        assertEquals(10, c.getValue());  
    }  
  
    @Test  
    void testOKConstructor1b() {  
        Counter c = new Counter(27);  
        assertEquals(27, c.getValue());  
    }  
  
    @Test  
    void testReset() {  
        Counter c = new Counter();  
        assertEquals(1, c.getValue());  
        c.reset();  
        assertEquals(0, c.getValue());  
    }  
}
```

```
@Test  
void testInc0() {  
    Counter c = new Counter();  
    assertEquals(1, c.getValue());  
    c.inc();  
    assertEquals(2, c.getValue());  
    c.inc();  
    c.inc();  
    assertEquals(4, c.getValue());  
}  
  
@Test  
void testIncKa() {  
    Counter c = new Counter();  
    assertEquals(1, c.getValue());  
    c.inc(1);  
    assertEquals(2, c.getValue());  
    c.inc(1);  
    c.inc(1);  
    assertEquals(4, c.getValue());  
}  
  
@Test  
void testIncKb() {  
    Counter c = new Counter();  
    assertEquals(1, c.getValue());  
    c.inc(2);  
    assertEquals(3, c.getValue());  
    c.inc(4);  
    c.inc(7);  
    assertEquals(14, c.getValue());  
}
```

```
@Test  
void testEquals() {  
    Counter c1 = new Counter(7);  
    Counter c2 = new Counter(7);  
    assertTrue(c1.equals(c2));  
    assertTrue(c2.equals(c1));  
    assertTrue(c1.equals(c1));  
}  
  
@Test  
void testNotEquals() {  
    Counter c1 = new Counter(7);  
    Counter c3 = new Counter(6);  
    assertFalse(c1.equals(c3));  
    assertFalse(c3.equals(c1));  
}  
  
@Test  
void testSame() {  
    Counter c1 = new Counter(7);  
    Counter c2 = c1;  
    assertTrue(c1==c2);  
    assertTrue(c1.equals(c2));  
}
```

Eseguire i test

- Per eseguire una classe di test, basta cliccarci sopra col tasto destro e scegliere «Run As» → «JUnit Test»

